Overview of Independent component analysis

Subject: Independent component analysis

1.

If we substitute a commonly used high-Kurtosis model pdf for the source signals p s = (1 − tanh ■ (s) 2) {\displaystyle p_{s}=(1-\tanh(s)^{2})} then we have

2.

where the A {\displaystyle A} is an m × m {\displaystyle m\times m} invertible matrix called the mixing matrix, s i {\displaystyle s_{i}} represents the m■dimensional vector containing the values of the sources at time t i {\displaystyle t_{i}} , and x i {\displaystyle x_{i}} is the corresponding vector of observed values at time t i {\displaystyle t_{i}} . The goal is to estimate both A {\displaystyle A} and the source signals { s i } {\displaystyle \{s_{i}\}} solely from the observed data { x i } {\displaystyle \{x_{i}\}} . After centering, the Gram matrix is computed as:

3.

Thus, ICA reduces to finding the orthogonal matrix V {\displaystyle V} . This matrix can be computed using optimization techniques via projection pursuit methods (see Projection Pursuit). Well-known algorithms for ICA include infomax, FastICA, JADE, and kernel-independent component analysis, among others. In general, ICA cannot identify the actual number of source signals, a uniquely correct ordering of the source signals, nor the proper scaling (including sign) of the source signals. ICA is important to blind signal separation and has many practical applications. It is closely related to (or even a special case of) the search for a factorial code of the data, i.e., a new vector-valued representation of each data vector such that it gets uniquely encoded by the resulting code vector (loss-free coding), but the code components are statistically independent.

4.

x i = a i , 1 s 1 + ■ + a i , k s k + ■ + a i , n s n {\displaystyle x_{i}=a_{i,1}s_{1}+\cdots +a_{i,k}s_{k}+\cdots +a_{i,n}s_{n}}

5.

G 1 = 1 a 1 log ■ (cosh ■ (a 1 u)) {\displaystyle G_{1}={\frac {1}{a_{1}}}\log(\cosh(a_{1}u))} and G 2 = − exp ■ (− u 2 2) {\displaystyle G_{2}=-\exp(-{\frac {u^{2}}{2}})}

6.

where ∧ {\displaystyle \wedge } is Boolean AND and ∨ {\displaystyle \vee } is Boolean OR. Noise is not explicitly modelled, rather, can be treated as independent sources. The above problem can be heuristically solved by assuming variables are continuous and running FastICA on binary observation data to get the mixing matrix G {\textstyle {\boldsymbol {G}}} (real values), then apply round number techniques on G {\textstyle {\boldsymbol {G}}} to obtain the binary values. This approach has been shown to produce a highly inaccurate result. Another method is to use dynamic programming: recursively breaking the observation matrix X {\textstyle {\boldsymbol {X}}} into its sub-matrices and run the inference algorithm on these sub-matrices. The key observation which leads to this algorithm is the sub-matrix X 0 {\textstyle {\boldsymbol {X}}^{0}} of X {\textstyle {\boldsymbol {X}}} where x i j = 0 , ∀ j {\textstyle x_{ij}=0,\forall j} corresponds to the unbiased observation matrix of hidden components that do not have connection to the i {\displaystyle i} -th monitor. Experimental results from show that this approach is accurate under moderate noise levels. The Generalized Binary ICA framework introduces a broader problem formulation which does not necessitate any knowledge on the generative model. In other words, this method attempts to decompose a source into its independent components (as much as possible, and without losing any information) with no prior assumption on the way it was generated. Although this problem appears quite complex, it can be accurately solved with a branch and bound search tree algorithm or tightly upper bounded with a single multiplication of a matrix with a vector.

7.

the kurtosis of the extracted signal y {\displaystyle \mathbf {y} } to be maximal precisely when y = s {\displaystyle \mathbf {y} =\mathbf {s} } . the kurtosis of the extracted signal y {\displaystyle \mathbf {y} } to be maximal when w {\displaystyle \mathbf {w} } is orthogonal to the projected axes S 1 {\displaystyle S_{1}} or S 2 {\displaystyle S_{2}} , because we know the optimal weight vector should be orthogonal to a transformed axis S 1 {\displaystyle S_{1}} or S 2 {\displaystyle S_{2}} . For multiple source mixture signals, we can use kurtosis and Gram-Schmidt Orthogonalization (GSO) to recover the signals. Given M signal mixtures in an M-dimensional space, GSO project these data points onto an (M-1)-dimensional space by using the weight vector. We can guarantee the independence of the extracted signals with the use of GSO. In order to find the correct value of w {\displaystyle \mathbf {w} } , we can use gradient descent method. We first of all whiten the data, and transform x {\displaystyle \mathbf {x} } into a new mixture z {\displaystyle \mathbf {z} } , which has unit variance, and z = (z 1 , z 2 , … , z M) T {\displaystyle \mathbf {z} =(z_{1},z_{2},\ldots ,z_{M})^{T}} . This process can be achieved by applying Singular value decomposition to x {\displaystyle \mathbf {x} } ,

8.

Nonlinear ICA The mixing of the sources does not need to be linear. Using a nonlinear mixing function f (· | θ) {\displaystyle f(\cdot |\theta)} with parameters θ {\displaystyle \theta } the nonlinear ICA model is x = f (s | θ) + n {\displaystyle x=f(s|\theta)+n} .

9.

Binary ICA A special variant of ICA is binary ICA in which both signal sources and monitors are in binary form and observations from monitors are disjunctive mixtures of binary independent sources. The problem was shown to have applications in many domains including medical diagnosis, multi-cluster assignment, network tomography and internet resource management. Let x 1 , x 2 , … , x m {\displaystyle {x_{1},x_{2},\ldots ,x_{m}}} be the set of binary variables from m {\displaystyle m} monitors and y 1 , y 2 , … , y n {\displaystyle {y_{1},y_{2},\ldots ,y_{n}}} be the set of binary variables from n {\displaystyle n} sources. Source-monitor connections are represented by the (unknown) mixing matrix G {\textstyle {\boldsymbol {G}}} , where g i j = 1 {\displaystyle g_{ij}=1} indicates that signal from the i-th source can be observed by the j-th monitor. The system works as follows: at any time, if a source i {\displaystyle i} is active (y i = 1 {\displaystyle y_{i}=1}) and it is connected to the monitor j {\displaystyle j} (g i j = 1 {\displaystyle g_{ij}=1}) then the monitor j {\displaystyle j} will observe some activity (x j = 1 {\displaystyle x_{j}=1}). Formally we have: